

Scalability Patterns Deep Dive

Phase 2 · Intermediate · Estimated time: 1.5 hours

1. Concept From First Principles

Phase 1 introduced horizontal and vertical scaling as ideas. This chapter goes one level deeper into the two patterns that actually make horizontal scaling of a database layer work in practice: **read replicas** and **connection pooling**. Both exist to answer the same underlying question — your single primary database can only handle so many simultaneous connections and so much query load before it becomes the bottleneck for your entire system, so how do you stretch it further before you're forced into sharding (Chapter 21)?

A **read replica** is a live, continuously-updated copy of your primary database. Every write (INSERT, UPDATE, DELETE) still goes to exactly one primary. The primary then streams those changes to one or more replicas, which apply them and stay in sync. Once a replica has an offer to serve, read-only queries (SELECT statements) can be pointed at any replica instead of hammering the primary. Since most real-world systems are read-heavy — a 90:10 or even 99:1 read-to-write ratio is common for content and catalog systems — this single change can multiply your effective read capacity by adding replicas, without touching a single line of business logic.

The catch is **replication lag**. Because the primary streams changes to replicas asynchronously in most setups, there's always a small delay — typically milliseconds, but sometimes seconds under heavy load — between a write landing on the primary and that same data appearing on a replica. If a user updates their profile and your application immediately reads their profile back from a replica that hasn't caught up yet, they'll see stale data. This is your first real brush with the consistency trade-offs that Chapters 18 and 19 explore in full depth — scalability and consistency are already in tension, even at this "simple" level.

Backend engineer analogy

This is exactly what Laravel's database configuration file supports natively when you define separate **'read'** and **'write'** connection arrays under a single connection name — Laravel's query builder automatically sends SELECTs to the read pool and everything else to the write connection. Node.js ORMs like Sequelize and Prisma support the identical pattern via replica-aware connection configs. If you've ever wired this up thinking "this just spreads out read traffic," you've already implemented this chapter's core idea in production.

Connection pooling solves a different, but related, problem. Every database connection is expensive to create: it involves a TCP handshake, often a TLS handshake, and an authentication round-trip, easily adding up to 50–200 milliseconds of pure overhead before your query even runs. If every incoming HTTP request to your Laravel or Express app opens a brand-new database connection and closes it when done, you pay that overhead on every single request, and worse, you can exhaust the database's hard limit on simultaneous connections (often just a few hundred) once your app server fleet grows to dozens of instances each opening connections independently.

A connection pool keeps a fixed set of already-open, already-authenticated database connections ready to hand out. When a request needs the database, it borrows a connection from the pool, uses it, and returns it — it never pays the connection-setup cost per request. Tools like PgBouncer (Postgres) and ProxySQL (MySQL) act as a proxy layer that pools connections on behalf of many application server instances, so instead of 5,000 individual app-server connections hitting the database directly, the database sees a stable, bounded 100–200 pooled connections, no matter how much your app-server fleet scales horizontally.

Combining both patterns

In a mature setup, you typically run two pools: a small primary pool for writes and any read that must be immediately consistent ("read-your-own-write" cases, covered in Chapter 18), and a larger replica pool for everything else, since reads usually dominate. The application layer needs a rule for which pool a given query goes to — most ORMs make this a configuration decision rather than something you hand-code per query, which keeps the business logic itself unaware of the underlying scaling strategy.

It's worth being explicit about what these two patterns do *not* solve. Read replicas only help with read scaling — they do nothing for write throughput, since every write still funnels through one primary. Connection pooling only reduces per-request overhead and protects the database from connection exhaustion — it doesn't add any new data capacity or write throughput either. When write volume itself becomes the bottleneck, no amount of replicas or pooling helps; that's precisely when sharding (splitting data itself across multiple independent primaries) becomes necessary, which is why this chapter is a deliberate stepping stone toward Chapter 21.

2. Real-World Example / Case Study

Flipkart's product catalog and listing pages are a textbook read-heavy workload: millions of shoppers browse and search product pages for every single purchase that happens, giving a read:write ratio that can run into the hundreds or thousands to one. Serving this entirely from a single primary database would be impossible; catalog reads are spread across a fleet of read replicas (often combined with aggressive caching in front of them, per Phase 1 Chapter 8), while writes — sellers updating stock or price — go through the primary and propagate outward.

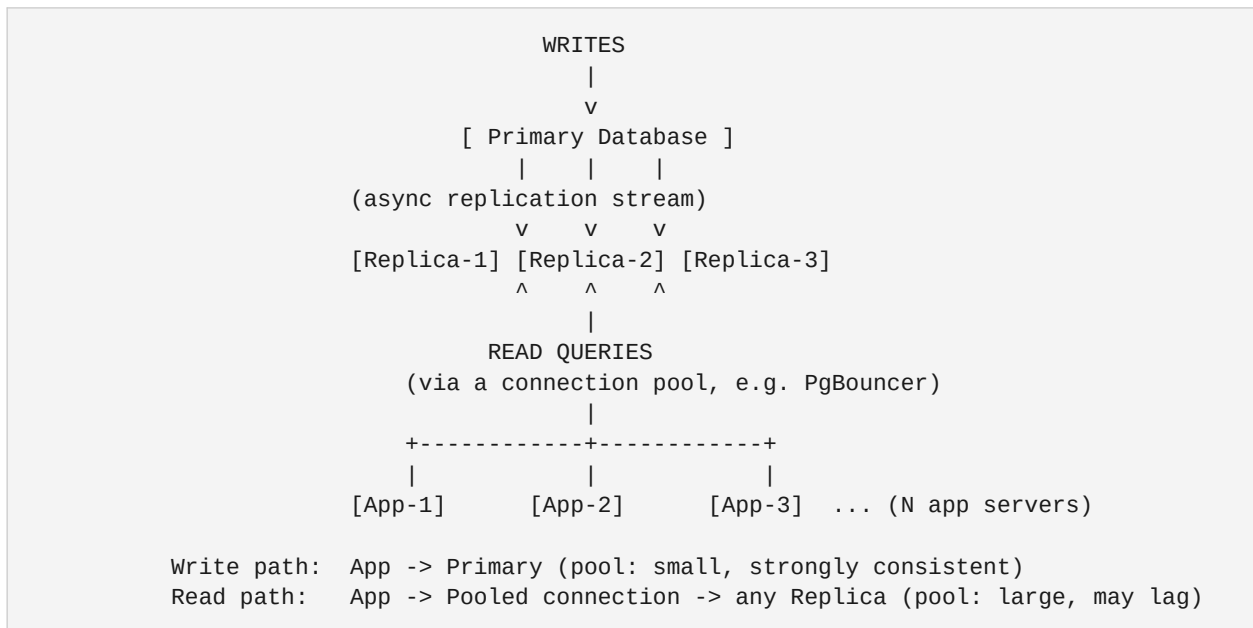
Swiggy's restaurant menu and availability pages follow the identical shape: a restaurant updates its menu or marks an item out of stock relatively rarely, but that menu is read by potentially thousands of customers browsing nearby restaurants every minute during peak lunch and dinner hours. Read replicas let Swiggy scale menu-serving capacity independently of how often restaurants actually change their menus.

On the connection pooling side, PhonePe and Razorpay, operating at very high transaction-per-second volumes across large fleets of stateless application servers, cannot afford every app-server instance to open its own direct database connections — at their scale, that alone would exhaust the database's connection limit before any real query load even arrives. A pooling layer (PgBouncer-style) sitting between the app fleet and the database is a near-mandatory piece of infrastructure once you're running more than a handful of app-server instances against one database.

In the Gulf market, Noon's product listing and search-suggestion pages follow the exact same read-heavy pattern as Flipkart's catalog, while Talabat's order-status polling (customers refreshing their delivery tracking page repeatedly) is a good example of a read pattern that benefits enormously from

replicas combined with caching, since order status changes far less often than customers refresh the page to check it.

3. Diagram



4. Trade-offs Table

Decision	Benefit	Cost / Risk
Add read replicas	Multiplies read capacity without app rewrite	Replication lag; stale reads possible
Route all reads to replicas	Maximizes primary headroom for writes	Read-your-own-write bugs if not handled
Add connection pooling (PgBouncer)	Removes per-request connect overhead; caps DB connections	One more infrastructure component to run and monitor
Skip pooling, connect directly per request	Simpler infra, fewer moving parts	Connection exhaustion as app fleet scales out
Separate read/write pools in app config	Clean separation of concerns; easy to reason about	Requires discipline — a misrouted write can silently target a replica

5. Common Interview Questions

Q1: "Your read-heavy API is slow under load. Walk me through how you'd scale it."

Model approach: Start with the cheapest fix first — caching (Phase 1 Chapter 8) — then add read replicas for anything cache misses can't absorb, and only reach for sharding once a single primary's write capacity itself is the bottleneck, not before. Interviewers want to see you reach for the least complex solution that solves the actual measured bottleneck.

Q2: "A user updates their name and immediately reloads the page, but sees the old name. Why, and how do you fix it?"

Model approach: Diagnose this as classic replication lag — the write hit the primary, the immediate read hit a replica that hasn't caught up. Fixes include routing that specific read to the primary right after a write from the same user session ("read-your-own-writes"), or having the client optimistically update its own UI instead of re-querying immediately.

Q3: "Why not just open a new database connection per request — what actually breaks?"

Model approach: Quantify the two costs — the 50–200ms handshake/auth overhead paid on every request, and the hard ceiling on simultaneous connections most databases enforce (often in the low hundreds), which a horizontally-scaled app fleet can blow through in seconds without pooling.

Q4: "Do read replicas help you scale writes?"

Model approach: No — be direct about this. Every write still funnels through exactly one primary regardless of how many replicas exist. Replicas only add read capacity; write-throughput scaling requires either a bigger primary (vertical scaling, with limits) or sharding (Chapter 21).

6. Practice Exercises

- 1 A news website gets 50,000 reads/second and 20 writes/second on articles. Sketch a database scaling plan using replicas and pooling, and justify how many replicas you'd start with.
- 2 Describe a concrete scenario in a food delivery app (like Swiggy) where a stale read caused by replication lag could produce a genuinely bad user experience, and propose a fix.
- 3 Your team disables connection pooling to 'simplify infrastructure' and traffic doubles the next month. Predict what fails first and why.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Master-Slave Replication System Design Fundamentals"	Clear walkthrough of the replica pattern this chapter builds on.
English	"#7 Master Slave Database System Design Fundamentals"	Reinforces read/write splitting with diagrams.
Hindi	"Postgres Connection Pooling Masterclass" — Coder's Gyan	Hands-on Hindi walkthrough of exactly how PgBouncer-style pooling works.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers scaling patterns as part of a full bilingual system design course.

8. Key Takeaways

- Read replicas multiply read capacity by copying data from one primary to one or more read-only followers.
- Replication is typically asynchronous, which introduces replication lag and possible stale reads.

- Connection pooling removes the per-request cost of opening a database connection and caps total connections.
- Neither read replicas nor connection pooling increase write throughput — only sharding does that.
- Mature systems route reads and writes through separate connection pools based on consistency needs.
- "Read-your-own-writes" is a common bug pattern caused by reading from a lagging replica right after a write.
- Always solve read-scaling with caching first, then replicas, before reaching for the complexity of sharding.